

Java Generics

Writing Type-Safe, Reusable Code

Learning Objectives

- What We Will Learn
 - Understand the problem Generics solve (Type Safety).
 - Define and implement **Generic Classes**.
 - Define and implement **Generic Methods**.
 - Use the **Diamond Operator** (<>).
 - Understand **Bounded Type Parameters**.
 - Master **Wildcards** (?, ? extends, ? super).
 - Understand **Type Erasure** and limitations.
 - Apply Generics to Collections.

The Problem (Pre-Generics)

- Why Do We Need Generics?
- **Before Java 5:** Collections held Object references.
- **Issues:**
 - **No Type Safety:** You could add a String to a list of Integers.
 - **Explicit Casting:** You had to cast objects back to their original type.
 - **Runtime Errors:** Errors appeared only when running the code (ClassCastException).

```
// Pre-Generics Code
List list = new ArrayList();
list.add("Hello");
list.add(100); // Compiles, but logic error

String s = (String) list.get(1); //ClassCastException at Runtime!
```

Introduction to Generics

- What Are Generics?
 - **Definition:** A way to parameterize types.
 - **Benefit:** Compile-time type checking.
 - **Syntax:** Angle brackets < >.
 - **Convention:** Use T (Type), E (Element), K (Key), V (Value).

```
// Generics Code
List<String> list = new ArrayList<>();
list.add("Hello");
list.add(100); // COMPILER ERROR!

String s = list.get(0); // No casting needed!
```

Generic Classes

- Creating a Generic Class
 - Allows a class to operate on objects of various types while providing type safety.
 - **Syntax:** `class ClassName<T> { ... }`

```
public class Box<T> {  
    private T content;  
  
    public void set(T content) {  
        this.content = content;  
    }  
  
    public T get() {  
        return content;  
    }  
}  
  
// Usage  
Box<Integer> intBox = new Box<>();  
intBox.set(10);  
Integer i = intBox.get();
```

Generic Methods

- Creating Generic Methods
 - Methods can have their own type parameters, independent of the class.
 - **Syntax:** <T> before the return type.

```
public class Util {  
    // Generic method  
    public static <T> void printArray(T[] array) {  
        for (T element : array) {  
            System.out.println(element);  
        }  
    }  
}  
  
// Usage  
Integer[] intArr = {1, 2, 3};  
String[] strArr = {"A", "B"};  
Util.printArray(intArr); // T becomes Integer  
Util.printArray(strArr); // T becomes String
```

Type Inference (Diamond Operator)

- The Diamond Operator <>
 - **Introduced:** Java 7.
 - **Feature:** The compiler infers the type from the context.
 - **Reduces Boilerplate:** No need to repeat types on the right side.

```
// Before Java 7
List<String> list = new ArrayList<String>();

// Java 7+ (Diamond Operator)
List<String> list = new ArrayList<>();
```

Bounded Type Parameters

- Restricting Types (extends)
 - Sometimes you want T to be a specific type or subclass.
 - **Syntax:** <T extends ClassName>
 - **Use Case:** When you need to call methods specific to that class.

```
// T must be a Number or subclass (Integer, Double, etc.)
public class NumberBox<T extends Number> {
    private T num;
    public double doubleValue() {
        // Safe to call because T extends Number
        return num.doubleValue();
    }
}
```


Wildcards (?)

- **Title:** Unbounded Wildcards
 - **Symbol:** ? (Question Mark).
 - **Meaning:** "Unknown type".
 - **Use Case:** When code doesn't depend on the specific type (e.g., reading a list).

```
public void printList(List<?> list) {  
    for (Object elem : list) {  
        System.out.println(elem);  
    }  
}
```

- **Restriction:** You **cannot** add elements to a List<?> (except null).
- **Reason:** The compiler doesn't know what type is safe to add.

Bounded Wildcards (PECS)

- Upper & Lower Bounded Wildcards
 - **? extends Type (Upper Bound):**
 - Represents Type or its subclasses.
 - **Producer:** You can **get** (read) items out.
 - **? super Type (Lower Bound):**
 - Represents Type or its superclasses.
 - **Consumer:** You can **put** (write) items in.
 - **Mnemonic: PECS** (Producer Extends, Consumer Super).

```
void process(List<? extends Number> list) {  
    Number n = list.get(0); // OK  
    list.add(1); // Compile Error  
}  
  
void addNumbers(List<? super Integer> list) {  
    list.add(10); // OK  
    Integer i = list.get(0); // Returns Object, not Integer  
}
```

Type Erasure

- How Generics Work Under the Hood
 - **Type Erasure:** Generics are a compile-time feature.
 - **Process:** The compiler replaces T with Object (or the bound) and inserts casts.
 - **Result:** At runtime, List<String> and List<Integer> are both just List.
 - **Limitations:**
 - Cannot create instances: new T() is illegal.
 - Cannot use primitives: List<int> is illegal (use List<Integer>).
 - Cannot use instanceof with generics: obj instanceof List<String> is illegal.

Generics in Collections

- Common Collection Generics
 - **List<E>**: Ordered collection.
 - **Set<E>**: Unique elements.
 - **Map<K, V>**: Key-Value pairs (Two type parameters).
 - **Queue<E>**: FIFO collection.

```
List<String> names = new ArrayList<>();  
Set<Integer> ids = new HashSet<>();  
Map<String, Integer> scores = new HashMap<>();
```

- **Note:** Always specify types to avoid raw type warnings.

Best Practices

- **Generics Best Practices**

- ☒ Do:

- Use Generics for all Collections.
 - Use Diamond Operator `<>`.
 - Use Wildcards (?) for API flexibility (PECS).

- ☒ Don't:

- Use Raw Types (List instead of List<Object>).
 - Use Generics with Primitives (List<int>).
 - Catch Generic Exceptions (catch (Exception<T> e) is invalid).
 - Create Generic Arrays (new T[10] is invalid).

Summary & Lab

- Summary:
 - Generics provide type safety and reduce casting.
 - Use `<T>` for classes/methods.
 - Use `?` extends for reading, `? super` for writing.
 - Remember Type Erasure limitations.
- Lab Exercise:
 - Create a generic class `Pair<K, V>` that holds two objects of different types.
 - Add `getKey()` and `getValue()` methods.
 - Test with `Pair<String, Integer>` and `Pair<Double, Boolean>`.
 - Challenge: Add a method `swap()` that returns a `Pair<V, K>`.